

Plan de test - Application de réservation

Ancrelieu

Objectif de ce document

Ce document cadre le travail de test sur l'application de réservation. Personne n'a jamais écrit de tests sur ce code : il faut donc décider par où commencer, et pourquoi.

Ce plan liste les domaines à tester en priorité, les niveaux de test recommandés pour chacun et les outils standards selon votre stack. Vous pouvez vous en écarter, à condition de justifier votre choix : ce plan est une base de discussion, pas une liste imposée.

Domaines métier à tester, par ordre de priorité

L'application de réservation repose sur trois familles de règles. Voici comment je les classe, du plus critique au moins urgent.

Domaine métier · Pourquoi c'est prioritaire · Niveau de test recommandé

Calcul du montant dû (tarification, remboursement sur annulation) · Impact financier direct ; c'est déjà la cause de l'incident client de cette semaine · Unitaire, en priorité absolue

Règles d'annulation (délais, conditions de refus) · Conditionne le calcul du remboursement ; source d'erreurs si la logique de délai est mal isolée · Unitaire

Accès aux données de réservation (un client ne voit que ses réservations) · Sujet sensible pour le client, remonté dans l'audit à venir · Intégration, puis vérifié à nouveau après la sécurisation

Parcours de réservation de bout en bout (recherche, disponibilité, confirmation) · C'est ce que l'utilisateur final vit réellement ; un bug ici se voit tout de suite · End-to-end (E2E)

Écrans d'administration et fonctionnalités annexes (export, statistiques) · Aucun impact financier ou de confidentialité direct à ce stade · Non prioritaire pour ce chantier

Niveaux de test recommandés

Trois niveaux de test structurent ce chantier. Un test unitaire vérifie une règle de calcul ou de décision isolée, sans dépendance externe. Un test d'intégration vérifie que plusieurs couches connectées (API, base de données) fonctionnent ensemble. Un test de bout en bout (E2E) vérifie qu'un parcours utilisateur complet fonctionne, de la recherche à la confirmation.

Ne testez pas tout au même niveau. Les règles de calcul et d'annulation se prêtent bien aux tests unitaires, rapides à exécuter et faciles à isoler. Les échanges entre l'API et la base de données se prêtent aux tests d'intégration. Le parcours de réservation complet se prête à un test E2E, mais un seul suffit pour ce chantier : ce n'est pas le niveau où on multiplie les cas.

Outils recommandés par stack

Utilisez les outils standards de votre stack plutôt que d'en introduire de nouveaux : ce sont ceux que l'équipe connaît déjà et qu'on retrouvera dans le projet de synthèse.

Stack · Tests unitaires · Doublures (mocks) · Tests d'intégration · Test E2E · Couverture de code

Java (Spring Boot) · JUnit 5 · Mockito · Spring Boot Test + Testcontainers · Selenium ou Cypress · JaCoCo

Python · PyTest · unittest.mock ou pytest-mock · PyTest (fixtures) + Testcontainers · Selenium · pytest-cov ou coverage.py

.NET · xUnit ou NUnit · Moq · WebApplicationFactory + Testcontainers · Playwright · Coverlet

Ce que ce chantier ne couvre pas

Ne partez pas sur une couverture exhaustive de l'application. Les écrans d'administration, l'export de données et les statistiques ne sont pas dans le périmètre de ce chantier : ils n'ont pas d'impact financier ni de risque de confidentialité identifié à ce stade.

Le déploiement et la maintenance ne sont pas non plus dans ce périmètre. Concentrez-vous sur le code et sur la preuve, par des tests, que les règles de calcul, d'annulation et d'accès aux données se comportent comme attendu.

Comment vous écarter de ce plan

Si vous choisissez de tester autre chose en priorité, ou de reporter l'un des points ci-dessus, notez la raison dans votre notebook ou vos commits. Une décision différente de ce plan n'est pas un problème si elle est justifiée par un risque métier réel ; une décision non expliquée le sera, en revanche, lors de la discussion avec votre mentor.